ITU-MiniTwit - Group e - Report

Frederik Hørup , Marie Johansen , Nikolej Lundquist , Sara Bagger , Vitus Brodersen <>

May 21, 2026

Contents

1 Introduction

Authors:

2 System

2.1 Architecture and Design

Authors:

2.2 Dependencies

Authors:

2.3 System States

Authors:

3 Process

3.1 CI/CD

Authors:

3.2 Monitoring

Authors:

3.3 Logging

Authors:

3.4 Security

Authors:

3.5 Availability and Scaling

Authors:

4 Reflection

Authors:

4.1 Evolution and refactoring

Authors: Frederik

For the simulator to work there needed to be added some new endpoints which were specified in the swagger documentation. To make the work easier the OpenAPI Generator CLI tool was used and the different endpoints specialized in what was expected of them. Furthermore, authorization using a basic token was implemented to set up for the different endpoints that need it.

The database for the application was a SQLite database running in the minitwit application. That meant that the persisted data the database was holding got deleted when a new deploy of the application occurred. There was therefore a need to migrate to a new database type so the data is persisted. It was chosen to migrate to a MySQL database which was hosted using Digital Ocean. This was chosen for its simplicity and time effectiveness for the group.

4.2 Operations

Authors: Frederik, Marie_

Checking monitoring and logs often to see if there is exes strain on the system and if any errors have been thrown. Furthermore check of the status page to see if any errors has been thrown by the simulator that the loging did not catch.

Database load

Figure 1: Database load

4.3 Maintenance

Authors: Fredrik

During the period after the simulator started many errors and faults were discovered. Registration for the users the simulator was making was not working as intended and authentication for the different endpoints did also not work properly. Furthermore, the return types and bodies for the endpoints were also not correct. To fix these issues, swarming was used to quickly discover where things were going wrong. When the fault? (better word please) had been found small teams were made to fix the different issues. For full report of the issues see #29, #32, #34 and #41

We were made aware that Grafana was returning an internal server error when trying to log in. It was discovered that Prometheus had filled the droplets storage so it could not try to access the internal volume to log in. To fix this a folder docker uses to write data to was deleted and the docker containers on the VM were restarted. To make sure this did not happen again storage retention was added Prometheus got its own volume. To see the whole bug report see issue #81.

After seeing that the application was down, the first thing that was done was check the logs. Here it could be seen that the reason for the application was due to thread pool starvation. Looking more at the logs it could be seen that a request to the application was taking upwards of 46 minutes. By looking at the database, it could be seen that it was fetching 200.000+ lines per second. It was therefore decided that the database needed to be indexed to combat this issue. After this was implemented and the application restarted it was up and running smoothly. The full bug report can be found on issue #99

Database load before indexing

4.4 Reflect and describe what was the "DevOps" style of work

Authors: Sara, Marie_

Compared to earlier software projects, this course introduced us to several DevOps practices that changed both our workflow and our understanding of software development. Many of these practices reflected the principles behind the Three Ways of DevOps(Kim et al. 2021): improving flow, enabling fast feedback, and encouraging continuous improvement through automation, monitoring, and maintenance. A major difference compared to earlier projects was the amount of automation involved in the workflow. Tasks such as testing, linting, building containers, generating reports, and deployment were automated through pipelines and scripts. This reduced repetitive manual work and improved consistency across the project.

Continuous Integration (CI) Automatically running tests/linting on every pull request was a major change compared to earlier projects. Previously, broken code would mainly be picked up manually during pull request reviews, which depended heavily on reviewers noticing issues. Now, with CI pipelines in place, running automatic tests, linting, static analysis tools etc. caught issues early with fast automated feedback. This improved confidence when merging code and reduced the risk of introducing bugs into production.

Continuous Deployment (CD) Automated deployment significantly improved the deployment process compared to earlier projects, where the steps toward deployment were manual and inconsistent. Using CD made our deployments faster, easily reproducable and less error-prone. At the same time, setting up this deployment infrastructure was not without faults.

Monitoring & Software Maintenance Monitoring through collecting logs and metrics made a big difference for us compared to earlier projects. Using tools such as Grafana and Prometheus gave us a much better understading of the system's runtime behaviour, and once set up correctly, the logs made a big difference in debugging. Identifying bottlenecks or failures that would otherwise have been difficult to detect, became significantly easier and made us focus more on maintaining the software in production, rather than only

focussing on implementing functionality. Reliability and stability became important parts of the development process rather than something considered only at the end.

5 Use of Generative AI

Authors: Nikolej

During the development of this project we used ChatGPT in two main ways:

Debugging. Often when encountering unexpected bugs and error messages, we would consult ChatGPT about the cause and potential fixes While it could rarely fix the issues entirely by itself, more often than not, it pointed us in the right direction.

Research. This course presents challenges involving numerous technologies, most of which we were unfamiliar with. As such, ChatGPT was used to summarize lengthy documentation and provide concrete guides tailored to our situation.

Generating boilerplate / configurations. ChatGPT was also used for simple tasks like generating boilerplate code or writing Github Actions Workflow files. For example, the `build-report.yml` workflow, that converts the markdown source into a pdf.

The use of AI has made these tasks easier, spared us many frustrations and saved consiberable time during development. On the flip side, AI may have deprived us the extensive knowledge and experience you gain by, for example, painstaking reading of documentation or spending hours resolving a tiny bug.

References

Kim, Gene, Jez Humble, Patrick Debois, John Willis, and Nicole Forsgren. 2021. *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. It Revolution.